

DenoiseRL: Bootstrapping Reasoning Models to Recover from Noisy Prefixes

Caijun Xu^{1,2}, Changyi Xiao¹, Zhongyuan Peng¹, Yixin Cao^{1,2,†},

¹Fudan University, ²Shanghai Innovation Institute

Abstract

Reinforcement learning has become a central paradigm for advancing reasoning in large language models, yet most existing methods still depend on stronger teacher models or heavily curated difficult datasets, limiting scalable capability improvement. In this paper, we introduce DenoiseRL, a reinforcement learning framework that substitutes external supervision with recovery-oriented optimization over failures from weak models. Instead of relying on stronger supervision or carefully engineered data, DenoiseRL learns directly from incorrect reasoning traces by converting them into opportunities for improvement, making training more scalable and less dependent on external resources. This yields a richer and more diverse learning signal, improving exploration efficiency from imperfect model behavior. As a result, DenoiseRL improves reasoning performance and overall training efficiency while reducing the need for expensive data curation or stronger teacher models. Empirically, DenoiseRL consistently outperforms strong on-policy RL baselines across competitive mathematical and general reasoning benchmarks and promotes stronger self-corrective behavior as training difficulty increases, highlighting an effective and scalable alternative pathway for improving reasoning in large language models.

Correspondence: cjxu25@m.fudan.edu.cn

1 Introduction

Reinforcement learning (RL) has emerged as a central post-training paradigm for large language models (LLMs), driving substantial advances in complex reasoning tasks [1, 8, 11, 22, 27, 34]. Despite these successes, state-of-the-art systems often rely on supervision or guidance from even stronger models [9, 39, 43]. This dependence exposes a structural limitation: when no sufficiently capable off-the-shelf teacher is available, further capability gains become increasingly difficult, raising a fundamental question: **how can strong models be obtained without relying on stronger models as supervisors?**

To address this challenge, prior work has explored two main directions. The first is the weak-to-strong paradigm, which improves stronger models using supervision derived from weaker ones [4, 7, 21]. While effective in practice, its performance is fundamentally constrained by the quality of the teacher signal and easily leads to noise in the training process [15, 38, 41]. The second direction focuses on increasing task difficulty through data construction [20, 35, 42], including harder problem synthesis, adversarial examples, and longer reasoning trajectories. However, these approaches typically depend on carefully engineered pipelines, complex filtering and verification procedures, and substantial human effort in data design and

^{*}Equal Contribution.

[†]Corresponding authors.

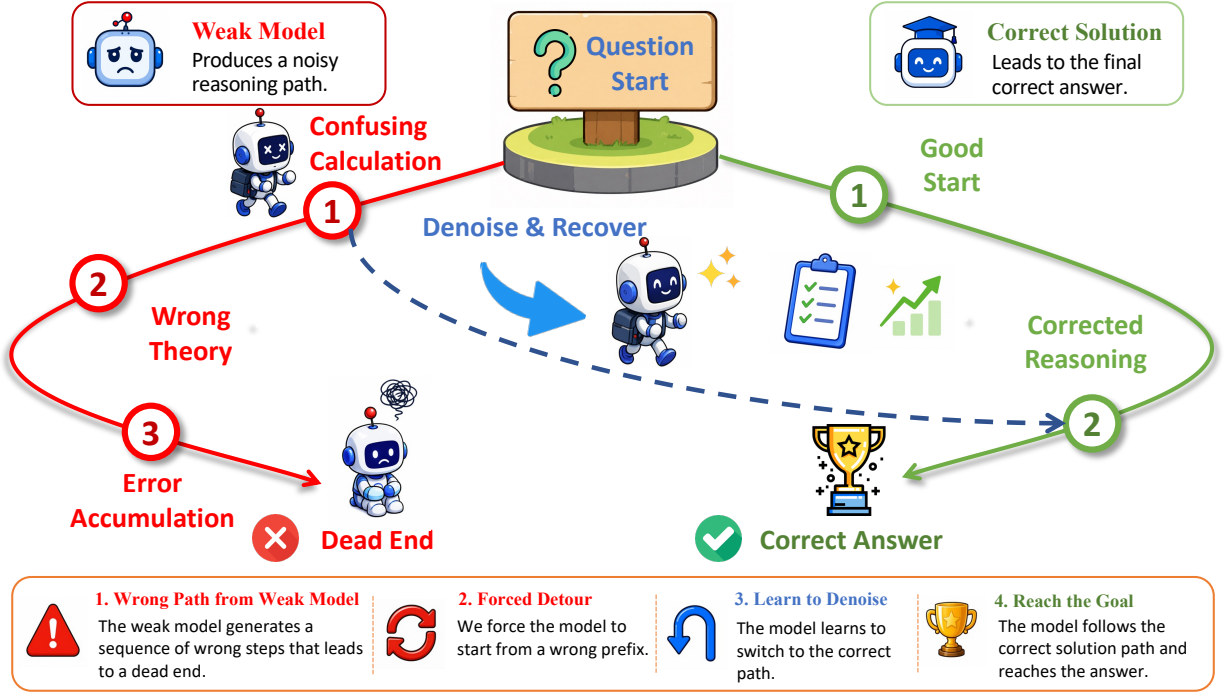


Figure 1 Schematic illustration of **DenoiseRL**. A weak model first generates an incorrect solution path. We then condition the policy model on a truncated prefix of this wrong trajectory, guiding it to continue from an erroneous reasoning state rather than imitating the weak model as a teacher. RL trains the policy to recover from this dead-end prefix—revising its reasoning, switching onto the correct solution path, and reaching the verified answer.

curation.

In this work, we propose **DenoiseRL**, a new RL paradigm that unifies weak-to-strong learning with difficulty-driven data synthesis. Instead of using weak models to synthesize hard data or provide learning signals, we repurpose weak models as generators of structured perturbations, which automatically increases training difficulty without generating new data. This enables scalable improvement of reasoning capability without reliance on external supervision and manually curated hard datasets. It also casts reasoning RL as a denoising problem: weak-model errors serve as structured corruptions of the reasoning trajectory, and the policy learns to reconstruct a valid solution path from these corrupted states, echoing the principle of denoising autoencoders and BART-style pretraining [17, 32].

Specifically, as illustrated in Figure 1, we sample noisy reasoning prefixes from weak models and inject them into the policy’s rollouts [25]. The policy is then optimized to denoise these corrupted prefixes and complete the reasoning trajectory correctly. We choose the prefix as the injection point because it plays a disproportionate role in shaping the subsequent reasoning trajectory. Prior work shows that high-quality prefixes can steer the policy toward more favorable reasoning states and improve RL efficiency through prefix-level conditioning or optimization [6, 29]. DenoiseRL reverses this perspective: we inject erroneous weak-model prefixes as structured noise, thereby controlling the starting state of reasoning and forcing the policy to recover from corrupted intermediate states. This mechanism induces two tightly coupled effects. First, it substantially expands the diversity of training states, since noisy prefixes span a much broader space of failure modes than correct trajectories, exposing the policy to off-policy contexts that are rarely encountered in standard on-policy RL [5, 14, 36]. Second, it directly strengthens a critical yet underdeveloped capability: recovery from mistakes. Rather than continuing along incorrect intermediate conclusions, the model is required to explicitly revise and correct its reasoning. By embedding erroneous prefixes into the optimization objective, DenoiseRL elevates self-correction from an emergent behavior to a direct training target [12, 33].

In our experiments, we find two important design lessons. First, noise should not be made arbitrarily strong: overly long erroneous prefixes can push the model into overthinking, with longer self-correction loops and increased uncertainty during reasoning. Second, updating the off-policy prefix leads to training instability, consistent with the recent observation [16, 26] that PPO-style objectives are sensitive to heavily off-policy tokens.

We summarize our contributions as follows:

- We propose DenoiseRL, a denoise-based RL paradigm that uses weak-model errors as noisy prefixes and trains stronger policies to reason out of them.
- We show that DenoiseRL consistently improves GRPO and DAPO across competitive mathematics and general reasoning benchmarks.
- We analyze key design factors in denoise training, including off-policy prefix masking, recovery intensity, and prefix-induced overthinking.

2 Related Work

Bootstrapping Reasoning via On-Policy RL. Outcome- and process-driven RL have superseded supervised fine-tuning for scaling reasoning capabilities [8, 13, 22]. Frameworks such as GRPO [27] and DAPO [40] drive this progress, yet they are fundamentally bounded by the model’s self-generated state distribution. Once the policy saturates, it predominantly generates correct rollouts or narrowly confined failure modes, creating an exploration bottleneck where informative failures are too scarce for meaningful gradient updates [5, 14, 19].

Weak-to-Strong Generalization (W2SG). To break capability plateaus, W2SG [4, 7, 28] leverages weaker models to supervise highly capable students. However, this paradigm inherently caps the student’s ceiling: the strong policy is optimized to imitate pseudo-labels, making it vulnerable to the weak supervisor’s noise and limited capacity [38]. Rather than treating the weak model as an imperfect oracle, DenoiseRL inverts its role, utilizing it strictly as a low-cost generator of out-of-distribution mistakes.

Prefix-Conditioned and Off-Policy Exploration. Another line of work improves exploration by injecting external prefixes or off-policy trajectories into RL. LUFFY [36] mixes off-policy reasoning traces with on-policy RL, while PrefixRL [25] conditions on successful off-policy prefixes and optimizes the remaining continuation. More broadly, prefix- and trajectory-guided methods use expert solutions, oracle hints, successful traces, or failure states to make sparse-reward problems more reachable [14, 23, 29]. DenoiseRL differs by using weak-model incorrect prefixes not as demonstrations or privileged hints, but as misleading reasoning states from which the policy must recover.

3 Method

We propose DenoiseRL, a denoising reasoning framework that trains LLMs to recover from incorrect intermediate reasoning states. Section 3.1 formally introduces the denoising reasoning task and its prefix-conditioned generation. Section 3.2 presents the RL objective for training denoise behavior.

3.1 Denoising Reasoning

Existing reasoning-oriented RL methods mainly improve performance by scaling supervision quality, such as relying on stronger teacher models or carefully curated hard examples. However, obtaining such supervision is often expensive and difficult to scale. Moreover, real reasoning processes frequently involve incorrect intermediate states that require correction and recovery, while this capability is not explicitly trained in standard RL. This motivates the need for a training paradigm that explicitly teaches models how to recover from noisy reasoning trajectories.

To this end, we introduce denoising reasoning, which treats incorrect intermediate reasoning states from a weak model as a form of structured noise. Concretely, an incorrect partial solution is prepended to the policy’s generation, and the policy is trained to continue reasoning from this corrupted state toward the correct answer. Under this framework, erroneous prefixes serve both as weak-to-strong perturbation signals and as a low-cost mechanism for increasing reasoning difficulty.

Specifically, we sample candidate solutions from a weak model and keep the ones the verifier judges as wrong. This is done once as an offline pre-processing step over the training set $\mathcal{D}$, so the pool $\{\mathcal{W}(q)\}_{q \in \mathcal{D}}$ is fixed throughout RL training and incurs no additional cost per step.

For problems on which π_w never produces a well-formed wrong answer in M trials, the pool $\mathcal{W}(q)$ is empty. In this case the denoise slots of q are instead replaced by additional standard main rollouts.

3.2 Reinforcement Learning for Recovering from Noisy Prefixes

In order to train the model to acquire the capability of denoising, each training step samples denoise rollouts in addition to standard on-policy rollouts, corresponding to the training objective of recovering from noisy prefixes. Let a question $q \sim \mathcal{D}$ be sampled from the training set, and let $\pi_\theta(\cdot | \cdot)$ be the policy model we optimize. We define:

- **Main rollouts** (N per problem) are standard on-policy rollouts:

$$y \sim \pi_\theta(\cdot | q). \quad (1)$$

- **Denoise rollouts** (K per problem) start from a partial noisy prefix. We draw an incorrect solution $w \sim \mathcal{W}(q)$ and, under a fixed prefix-ratio strategy with hyperparameter $\rho \in (0, 1]$, retain its first

$$p = \max\left(1, \lfloor \rho |w| \rfloor\right) \quad (2)$$

tokens as an assistant message $w_{1:p}$. The policy then continues writing from this off-policy prefix:

$$y_{>p} \sim \pi_\theta(\cdot | q, w_{1:p}). \quad (3)$$

Output budget and folding. As both rollout types should share the same response window of width R for a fair comparison and the prefix $w_{1:p}$ already consumes p tokens of that budget, denoise rollouts are folded so that the visible response is

$$\tilde{y} = \left[\underbrace{w_{1:p}}_{\text{prefix}}, \underbrace{y_{p+1:p+L}}_{\text{continuation}} \right], \quad p + L \leq R, \quad (4)$$

where we denote by R the maximal response length, $T_y \leq R$ is the number of generated tokens, and $L = \min(T_{y_{>p}}, R - p)$ is the length of the kept continuation and the trailing p tokens beyond the length-fair budget are discarded. The verifier is applied to the complete folded response $\tilde{y}$, and therefore assigns the reward $r(\tilde{y}; q)$ based on whether the policy successfully reaches the correct answer after conditioning on the noisy prefix. During training, we also only update the on-policy continuation part $y_{p+1:p+L}$ to train the model to recover from noisy prefixes.

Token-level GRPO objective. Since denoise rollouts provide negative samples for problems that are easy to handle, allowing positive samples to carry effective learning signals, we use a Group-Relative Policy Optimization (GRPO) [27] advantage that shares its baseline across all $N + K$ trajectories of the same problem. Let $\mathcal{G}(q) = \{1, \dots, N + K\}$ index the group of rollouts associated with q , with terminal rewards $r_i \in \{0, 1\}$. The trajectory-level advantage is

$$A_i = \frac{r_i - \mu_q}{\sigma_q + \varepsilon}, \mu_q = \frac{1}{N+K} \sum_{j \in \mathcal{G}(q)} r_j, \sigma_q^2 = \frac{1}{N+K} \sum_{j \in \mathcal{G}(q)} (r_j - \mu_q)^2. \quad (5)$$

Letting $c_{i,t}$ denote the context of token i, t ($c_{i,t} = q$ for a main rollout and $c_{i,t} = (q, w_{1:p_i})$ for a denoise rollout), the per-token importance ratio is

$$r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t} \mid c_{i,t}, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid c_{i,t}, y_{i,<t})}, \quad (6)$$

and the trajectory-level clipped surrogate [24] is

$$\mathcal{L}_i^{\text{PPO}}(\theta) = \frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} \min(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_{i,t}). \quad (7)$$

Joint objective over the two distributions. Our final objective is a joint expectation over the problem distribution and the two rollout distributions. Writing $\pi_\theta^{\text{main}}(\cdot \mid q) = \pi_\theta(\cdot \mid q)$ and $\pi_\theta^{\text{denoise}}(\cdot \mid q, w) = \pi_\theta(\cdot \mid q, w_{1:p})$ for the two response-generating distributions, population objective is

$$\mathcal{J}(\theta) = \frac{N}{N+K} \mathcal{J}_{\text{main}}(\theta) + \frac{K}{N+K} \mathcal{J}_{\text{denoise}}(\theta), \quad (8)$$

where the two objective components are defined as:

$$\mathcal{J}_{\text{main}}(\theta) = \mathbb{E}_{\substack{q \sim \mathcal{D} \\ y \sim \pi_{\theta_{\text{old}}}^{\text{main}}(\cdot \mid q)}} [\mathcal{L}^{\text{PPO}}(\theta; q, y)], \quad (9)$$

$$\mathcal{J}_{\text{denoise}}(\theta) = \mathbb{E}_{\substack{q \sim \mathcal{D}, w \sim \mathcal{W}(q) \\ y \sim \pi_{\theta_{\text{old}}}^{\text{denoise}}(\cdot \mid q, w)}} [\mathcal{L}^{\text{PPO}}(\theta; q, w_{1:p}, y)]. \quad (10)$$

This formulation can be interpreted as optimizing the policy under a mixture training distribution: the model is simultaneously encouraged to solve problems from scratch and to recover from corrupted intermediate reasoning states.

The Monte-Carlo estimator we actually optimize at every step is the natural sample average of $\mathcal{J}(\theta)$, defined as:

$$\hat{\mathcal{J}}(\theta) = \frac{1}{B(N+K)} \sum_{b=1}^B \left[\sum_{i \in \mathcal{M}(q_b)} \mathcal{L}_i^{\text{PPO}}(\theta) + \sum_{i \in \mathcal{S}(q_b)} \mathcal{L}_i^{\text{PPO}}(\theta) \right]. \quad (11)$$

Here, B is the number of problems per batch, $\mathcal{M}(q_b)$ denotes the N main rollouts and $\mathcal{S}(q_b)$ denotes the K denoise rollouts of q_b . The two rollout types thus contribute as a mixture weighted by $N/(N+K)$ and $K/(N+K)$, share a single advantage baseline within each problem, and only ever update the policy on tokens it generated itself.

4 Experiments

4.1 Settings

Noisy Prefix Collection. We use Qwen2.5-1.5B-Instruct [37] as the weak model to collect the incorrect reasoning trajectory, sampling the model on MATH-7.5K [10] with 8 rollouts per question, obtaining the incorrect ones after filtering.

Reinforcement learning. We train Qwen3-4B-Base and Qwen3-8B-Base [31] as policy models on MATH-7.5K. For our recovery training, each problem is sampled with $N = 12$ standard on-policy rollouts and $K = 4$

Table 1 Main results on mathematical and reasoning benchmarks. For AMC23, AIME2024, and AIME2025, we report AVG@16; for MATH500 and BBEH, we report AVG@1. Results are grouped by base model. Within each group, the best result is shown in **bold** and the second-best result is underlined. Tied second-best results are all underlined.

Method	MATH500	AMC23	AIME24	AIME25	BBEH	Avg.
<i>Qwen3-4B-Base</i>						
Base	70.0	43.1	8.3	7.7	4.1	26.6
GRPO	83.6	<u>63.1</u>	<u>22.1</u>	18.1	11.1	39.6
DAPO	83.8	62.5	20.6	21.5	10.4	39.8
DenoiseRL-GRPO	85.8	61.4	24.8	23.3	<u>14.8</u>	42.0
DenoiseRL-DAPO	<u>84.6</u>	63.6	21.9	<u>21.7</u>	15.7	<u>41.5</u>
<i>Qwen3-8B-Base</i>						
Base	70.4	49.2	11.9	10.8	4.1	29.3
GRPO	<u>87.8</u>	69.7	24.0	22.9	10.6	43.0
DAPO	87.0	69.7	23.8	21.7	<u>11.7</u>	42.8
DenoiseRL-GRPO	87.2	<u>70.3</u>	<u>24.6</u>	<u>23.1</u>	11.5	<u>43.3</u>
DenoiseRL-DAPO	88.2	71.4	27.0	24.8	12.6	44.8

denoise rollouts with 4096 tokens as the response length. Denoise rollouts are initialized with noisy prefixes and we use a fixed prefix ratio $\rho = 0.2$. For all runs, we use a prompt batch size of 16, learning rate 10^{-6} , no KL loss or length loss. The PPO [24] clipping range is set as $\epsilon_{\text{low}} = \epsilon_{\text{high}} = 0.2$. During training, we sample with temperature= 1.0 and top-p= 1.0. The training prompt is shown in Appendix A.

Evaluation. We evaluate models on mathematical reasoning benchmarks including MATH500 [18], AMC23 [3], AIME2024, AIME2025 [2], and BBEH [30]. Validation decoding uses temperature = 0.6, top-p= 0.95. We report AVG@16 for AIME2024, AIME2025, and AMC23, and AVG@1 for the remaining benchmarks.

Baselines. We compare our recovery training against the base model and two RL baselines, GRPO [27] and DAPO [40] on Qwen3-4B-Base and Qwen3-8B-Base.

4.2 Results

Table 1 reports the main results on two model scales and two RL backbones. DenoiseRL consistently improves the average performance of both GRPO and DAPO across Qwen3-4B-Base and Qwen3-8B-Base, showing that the proposed recovery training is not tied to a specific model size or optimization backbone.

On Qwen3-4B-Base, DenoiseRL-GRPO improves the GRPO baseline from 39.6% to 42.0% average score, with clear gains on MATH500, AIME2024, AIME2025, and BBEH. DenoiseRL-DAPO also improves over DAPO, increasing the average score from 39.8% to 41.5%. Notably, DenoiseRL-DAPO achieves the best performance on AMC23 and BBEH, while DenoiseRL-GRPO obtains the best overall average. These results indicate that denoise rollouts provide complementary training signals to standard RL, especially on harder reasoning benchmarks.

The same trend holds on Qwen3-8B-Base. DenoiseRL-GRPO improves the GRPO baseline from 43.0% to 43.3%, achieving the second best average performance among all 8B models. Meanwhile, DenoiseRL-DAPO improves DAPO from 42.8% to 44.8% and achieves the best result on every evaluated benchmark. Overall, the improvements across both model scales and both RL backbones suggest that DenoiseRL is a general training strategy for enhancing reasoning models, rather than an isolated gain under a single experimental setting.

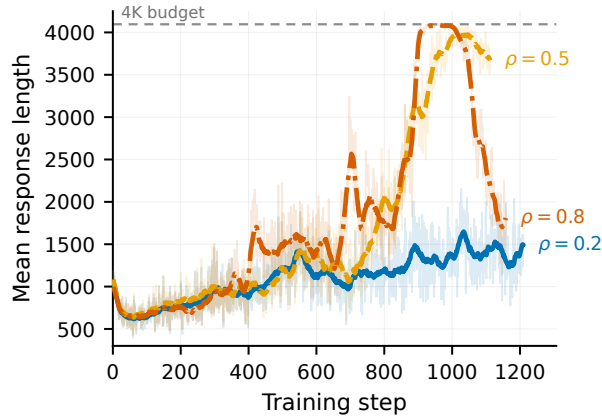


Figure 2 Mean response length under different prefix ratios with $K = 4$ denoise rollouts. Larger prefix ratios induce longer generations and more frequent length spikes, indicating overthinking during recovery.

Table 2 Effect of length-fair output budget ($p+L \leq R$) on Qwen3-4B-Base at $K=4$, $\rho=0.2$. Bold entries are the better score per benchmark.

Folding mode	MATH500	AMC23	AIME2024	AIME2025	BBEH	Average
Length-fair	85.8	61.4	24.8	23.3	14.8	42.0
No length cap	84.2	60.6	18.8	24.2	13.5	40.2

4.3 Intensity of Noise

In order to discover how the intensity of noise affects the policy model during training, in this experiment, we conduct experiments on Qwen3-4B-Base and GRPO RL backbone with two hyperparameters: the prefix ratio ρ and the number of denoise rollouts K .

Figure 2 compares $\rho \in \{0.2, 0.5, 0.8\}$ with the same number of denoise rollouts ($K = 4$). The mild setting $\rho = 0.2$ stays relatively compact, with an average response length of 1.38K tokens over the last 100 training steps. In contrast, $\rho = 0.8$ increases the same statistic to 2.26K tokens and reaches the 4096-token budget during training. The intermediate setting $\rho = 0.5$ is also unstable in length, averaging 3.87K tokens over the last 100 steps. Through careful examination, we discovered an interesting empirical phenomenon: a larger prefix-truncation ratio ρ induces more pronounced overthinking behavior, endless self doubt and verification, as shown in figure 3. As a very high noisy ratio ρ can cause the trajectory to deviate too far from the correct answer, the model becomes more skeptical of its own response. Once it reaches a plausible answer, it may continue to verify, rewrite, or restart the derivation process instead of stopping.

We next vary the number of denoise rollouts $K \in \{1, 4, 8\}$ while fixing $\rho = 0.2$. To keep the per-step sampling budget comparable, we reduce the number of standard on-policy rollouts as K increases, so larger K implies a higher fraction of denoise trajectories in each batch.

Figure 4 summarizes the downstream effect. With only $K = 1$ denoise rollout per problem, the corrective signal is too sparse to reliably teach recovery: the average gain is +14.9%. At the other extreme, $K = 8$ allocates half of the sampled trajectories to recovery, which distracts from the learning signals to solve problems and yields the weakest overall result, with +11.9% average gain. The balanced setting $K = 4$ achieves the best trade-off, with the highest average gain of +16.3% and the strongest peaks on the hardest benchmarks, including AIME24 +16.5% and AIME25 +16.9%. These results indicate that recovery intensity must be tuned jointly with the standard RL objective: too small K provides little benefit, whereas too large K shifts optimization away from the core goal of problem solving.

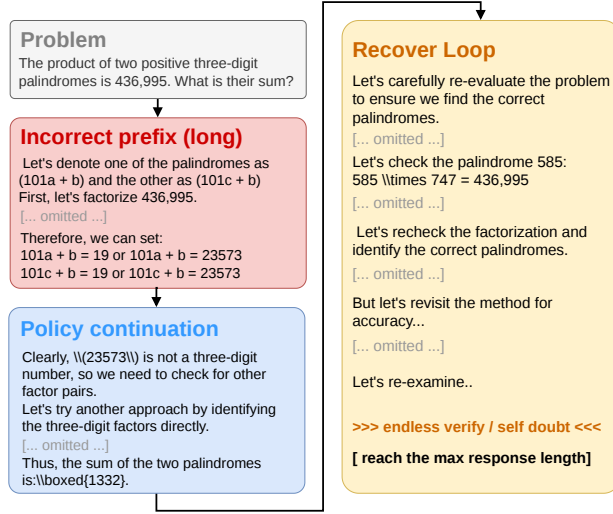


Figure 3 A representative high-prefix rollout at step 1000 ($\rho = 0.8$, $K = 4$). The retained noisy prefix misidentifies digit positions; the policy continues it to a wrong answer, then repeatedly revisits and re-checks the derivation instead of stopping.

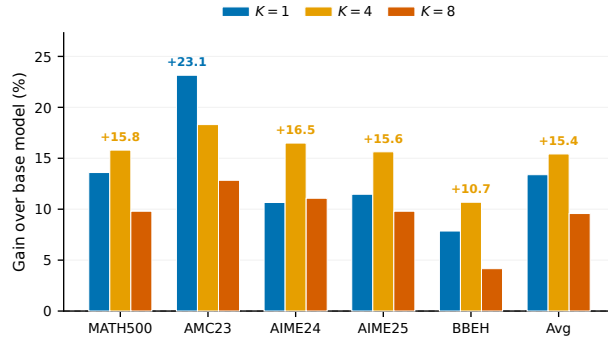


Figure 4 Benchmark gains over Qwen3-4B-Base with GRPO under different numbers of denoise rollouts with $\rho = 0.2$. $K = 4$ yields the strongest overall improvement; $K = 1$ under-supervises recovery, while $K = 8$ over-emphasizes it and hurts the primary objective of solving problems from scratch.

4.4 Off-policy Prefix

As the complete reasoning trajectory consists of the off-policy prefix and the model continuation, this ablation asks whether we should also update the off-policy prefix. Our default DenoiseRL setup only backpropagates through the on-policy continuation tokens in denoise rollouts: the offline noisy prefix $w_{1:p}$ is visible to the reward verifier but masked out of the PPO loss. Concretely, we set the prefix tokens' response mask to 1 so that gradients flow through the entire folded response $\tilde{y} = [w_{1:p}, y_{p+1:p+L}]$.

Figure 5 shows that updating the off-policy prefix is unstable. Validation accuracy improves early in training, peaking at 34.7% average at step 80, but then degrades sharply after step 140 and collapses to 0 on all benchmarks by step 400. In parallel, mean response length first shrinks to roughly 450 tokens, then spikes and saturates at the 4096-token budget.

We attribute this failure to a large mismatch between the log-probability distribution of the offline noisy prefix under the current policy and under the behavior policy that produced it. Applying PPO ratios (6) to these heavily off-policy tokens injects noisy, high-variance gradient updates that destabilize both reasoning quality and length control, consistent with prior work on RL for language models [16, 26].

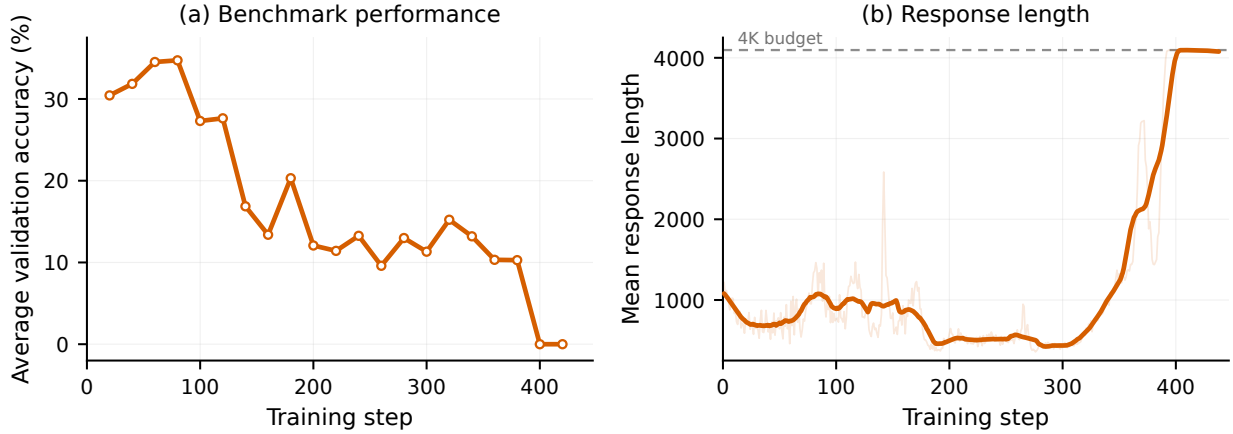


Figure 5 Training collapse when PPO updates are applied to off-policy noisy prefixes. **(a)** Average validation accuracy across MATH500, AMC23, AIME24, AIME25, and BBEH. **(b)** Mean response length (25-step moving average).

Table 3 Average training time per training step on Qwen3-4B-Base.

Method	Rollouts / problem	Time (s / step)
GRPO baseline	16 on-policy	43.8
DenoiseRL-GRPO	12+4	49.7

4.5 Fairness of Output Budget

To keep comparison with main rollouts fair, our default folding enforces $p + L \leq R$ (Eq. (4)): once the prefix consumes p tokens, the kept continuation is truncated to at most $R - p$ tokens. In order to investigate the necessity of keeping the fairness of output budget, this ablation preserves the full prefix and all R generated tokens, so a denoise rollout can expose up to $p + R$ tokens in total.

Table 2 shows that the length-fair design is effective. Without the budget cap, denoise rollouts receive extra generation capacity beyond the R -token window shared by main rollouts, which weakens the overall result by 1.8 percentage points on average (42.0% vs. 40.2%). The gap suggests that an unfairly long recovery budget encourages verbose but less reliable reasoning. Enforcing $p + L \leq R$ keeps both rollout types on equal footing and yields the stronger performance.

4.6 Training Time Efficiency

To study the time efficiency of our method, we report training time per optimizer step on Qwen3-4B-Base with MATH-7.5K and batch size 16 on 4×H100, recorded from the same infrastructure as our main runs. DenoiseRL with $K=4$ and $\rho=0.2$ uses $N=12$ on-policy rollouts plus $K=4$ denoise rollouts per problem; the GRPO baseline samples 16 on-policy rollouts, so both methods keep a comparable per-step rollout budget.

Table 3 shows that DenoiseRL is slightly slower per step (49.7s vs. 43.8s). Figure 6 explains part of this gap: over the last 100 training steps, DenoiseRL generates 1.27× more continuation tokens than GRPO, because denoise rollouts enhance the model’s capabilities of rethinking and repairing reasoning. The folded trajectories are therefore longer to sample and backpropagate, which naturally increases wall-clock time even though the per-step rollout count matches GRPO. Despite this modest overhead, recovery training stays in the same cost regime and delivers higher downstream accuracy.

Table 4 A representative recovery case. Red text marks key errors in the noisy prefix, while bold text highlights recovery behavior in the model continuation.

Part	Content
Problem	A school has between 150 and 200 students enrolled. If one student is absent, the students can be divided equally into six sections. What is the sum of all possible enrollment numbers?
Noisy Prefix	Let the number of students be n . Since one student is absent, $n - 1$ is divisible by 6, so $n - 1 = 6k$ and $n = 6k + 1$. [omitted: several attempted substitutions for k] For $k = 17$, since $6 \times 17 + 1 = 103$, 103 is between 150 and 200. For $k = 18$, since $6 \times 18 + 1 = 109$, 109 is between 150 and 200.[omitted: more invalid candidates below 150] Therefore, the possible values for n are 193, 199, and the sum is 193 + 199 = 392 .
Model Continuation	Let's re-evaluate the problem step by step to ensure we find all possible values of n that satisfy the given conditions. Since $n - 1 \equiv 0 \pmod{6}$, we have $n \equiv 1 \pmod{6}$ and $n = 6k + 1$. Now, find all integers k such that $150 < n < 200$: $150 < 6k + 1 < 200$, hence $149 < 6k < 199$ and $\frac{149}{6} < k < \frac{199}{6}$, i.e., $24.833 \dots < k < 33.166 \dots$. Since k must be an integer, k ranges from 25 to 33 inclusive. The corresponding values of n are 151, 157, 163, 169, 175, 181, 187, 193, 199. Finally, $151 + 157 + \dots + 199 = 1575$, so the final answer is 1575 .

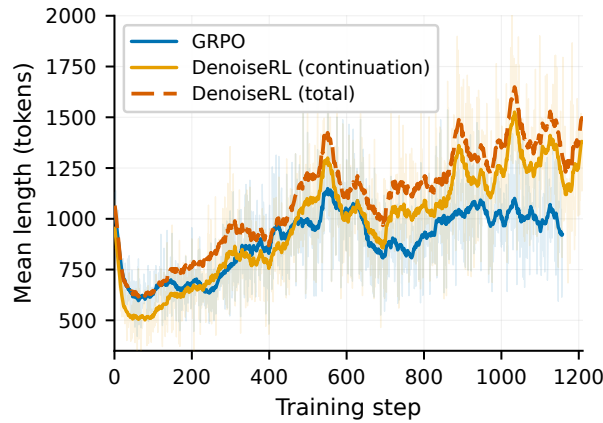


Figure 6 Mean rollout length during training. DenoiseRL continuation subtracts the mean offline prefix length; the dashed orange curve is the full folded response (prefix + continuation). GRPO has no prefix, so its curve is the full response length.

4.7 Case Study

The purpose of this case study is to examine whether DenoiseRL induces genuine denoising and recovery behavior rather than merely encouraging the policy to continue from a noisy prefix. In particular, we inspect a rollout where the prefix contains a partially correct derivation but reaches an incorrect answer due to faulty enumeration. As shown in Table 4, the model continuation does not follow the erroneous conclusion. Instead, it re-checks the core constraint, recomputes the feasible range, and repairs the final answer.

This suggests that denoise rollouts teach the model to use weak-model errors as perturbations: the policy learns to preserve useful partial reasoning while correcting the failure modes that lead to wrong answers. We provide more cases in Appendix B.

5 Conclusion

We propose DenoiseRL, a recovery-oriented RL framework that improves reasoning capability by training models to recover from incorrect intermediate reasoning trajectories generated by weak models. Instead of relying on stronger teachers or manually curated hard datasets, DenoiseRL converts weak-model failures into structured perturbations that continuously reshape the training distribution and increase reasoning

difficulty in a scalable manner. Our results demonstrate that recovery-centric optimization not only improves performance and training efficiency across challenging reasoning benchmarks, but also strengthens the model’s ability to revise and correct its own reasoning process. Furthermore, the emergence of stronger overthinking and self-correction behaviors under longer corrupted prefixes suggests that recovery training influences deeper reasoning dynamics beyond simple performance gains. Overall, this work provides a new perspective on scalable post-training for reasoning models by showing that model mistakes themselves can serve as a powerful source of learning signal.

Limitations

Although DenoiseRL demonstrates strong empirical performance, several limitations remain. First, the effectiveness of the generated perturbations still depends on the behavior of the weak models used for corruption generation. If the weak models produce errors that are overly trivial, repetitive, or unrealistic, the resulting recovery signal may provide limited training value and fail to induce meaningful reasoning improvements.

Second, although recovery-oriented training enhances the model’s self-correction capability, increasing the corruption length can also amplify overthinking behavior, resulting in unnecessarily long reasoning trajectories, higher inference cost, and reduced decoding efficiency. Balancing stronger recovery supervision with efficient reasoning therefore remains an important direction for future work.

A Prompt

Prompt z_q :

System

You are a helpful assistant.

User

{question}

Please reasoning step by step, and put your final answer in `\\boxed{}`.

B Supplementary Cases

We provide two additional recovery cases to further illustrate the behavior induced by DenoiseRL. For readability, we omit some repetitive intermediate calculations while preserving the key prefix-continuation boundary determined by the recorded token-level prefix length. Red text marks the error or failure mode in the noisy prefix, while bold text highlights recovery behavior in the model continuation.

The first case in Table 5 shows recovery from a misleading local probability calculation. The prefix contains useful structural information that there are two tribes of size 8, but it frames the problem around the probability of an individual quitter, which is not the most direct sample space. The continuation does not elaborate on this flawed direction. Instead, it explicitly changes strategy, counts unordered pairs of quitters, and compares favorable same-tribe pairs against all possible pairs. This behavior indicates that the policy can preserve useful problem facts from the prefix while rejecting an unproductive probabilistic framing.

The second case in Table 6 illustrates recovery from an inefficient search procedure. The prefix correctly transforms the intersection problem into a cubic equation, but then pursues explicit roots through repeated rational-root checks. This is unnecessary for the target quantity, since the question asks only for aggregate coordinate sums. The continuation identifies the dead end, switches to Vieta’s formulas, and obtains A directly from the missing x^2 coefficient. It then transfers this aggregate information back through the linear relation $y = 1 - \frac{x}{4}$ to compute B . Together with the main-text case, these examples suggest that denoise rollouts do more than encourage continuation from noisy contexts: they train the model to re-assess the current reasoning state, retain useful intermediate structure, and replace flawed or inefficient local strategies with solution-relevant ones.

Table 5 A supplementary recovery case on probability reasoning. The noisy prefix contains an incorrect local probability statement, while the continuation switches to a correct counting formulation.

Part	Content
Problem	The reality game show <i>Survivor</i> is played with 16 people divided into two tribes of 8. In the first episode, two people get homesick and quit. If every person has an equal chance of being one of the two quitters, what is the probability that both people who quit are from the same tribe?
Noisy Prefix	To determine the probability that both people who quit are from the same tribe, we can break the problem down into manageable steps. There are 16 people in the game, divided equally into two tribes of 8. For Tribe A, there are 8 people. The prefix then states that the probability that a specific person quits is $\frac{1}{15}$, because one person is chosen out of 16 and the remaining choices are considered. <i>[omitted: the prefix begins from a local individual-quitter calculation rather than the correct pair-level sample space.]</i>
Model Continuation	Since we are finding the probability of both quitters being from the same tribe, we need a different approach. The continuation constructs the correct sample space: $\binom{16}{2} = 120.$ The favorable cases are those where both quitters come from Tribe A or both come from Tribe B: $\binom{8}{2} + \binom{8}{2} = 28 + 28 = 56.$ Therefore, $\frac{56}{120} = \frac{7}{15},$ so the final answer is $\frac{7}{15}$

Table 6 A supplementary recovery case on algebraic reasoning. The noisy prefix attempts explicit root search, while the continuation recovers by using Vieta’s formulas.

Part	Content
Problem	The graphs of $y = x^3 - 3x + 2$ and $x + 4y = 4$ intersect in the points (x_1, y_1) , (x_2, y_2) , and (x_3, y_3) . If $x_1 + x_2 + x_3 = A$ and $y_1 + y_2 + y_3 = B$, compute the ordered pair (A, B) .
Noisy Prefix	From $x + 4y = 4$, the prefix derives $y = 1 - \frac{x}{4}.$ Substituting this into $y = x^3 - 3x + 2$ gives $1 - \frac{x}{4} = x^3 - 3x + 2,$ and hence $4x^3 - 11x + 4 = 0.$ The prefix then attempts to find the individual roots using the Rational Root Theorem, testing candidates such as $x = 1$, $x = \frac{1}{2}$, $x = -\frac{1}{2}$, $x = 2$, and $x = -1$. <i>[omitted: several direct substitutions showing that these candidates are not roots.]</i> The prefix keeps searching for explicit roots, although the problem only asks for the sums of the coordinates.
Model Continuation	It seems the rational root theorem isn’t yielding roots easily. Let’s try another method. Given $4x^3 - 11x + 4 = 0,$ the continuation writes it as $4x^3 + 0x^2 - 11x + 4 = 0.$ By Vieta’s formulas, $x_1 + x_2 + x_3 = -\frac{0}{4} = 0,$ so $A = 0$. Using $y = 1 - \frac{x}{4}$, $y_1 + y_2 + y_3 = \left(1 - \frac{x_1}{4}\right) + \left(1 - \frac{x_2}{4}\right) + \left(1 - \frac{x_3}{4}\right) = 3 - \frac{x_1 + x_2 + x_3}{4} = 3.$ Thus, the final answer is 0, 3

References

- [1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024. URL <https://arxiv.org/abs/2402.14740>.
- [2] Art of Problem Solving. Aime problems and solutions, 2025. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.
- [3] Art of Problem Solving. Amc problems and solutions, 2025. URL https://artofproblemsolving.com/wiki/index.php?title=AMC_Problems_and_Solutions.
- [4] Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, et al. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. *arXiv preprint arXiv:2312.09390*, 2023.
- [5] Yuzheng Cai, Siqi Cai, Yuchen Shi, Zihan Xu, Lichao Chen, Yulei Qin, Xiaoyu Tan, Gang Li, Zongyi Li, Haojia Lin, et al. Training-free group relative policy optimization. *arXiv preprint arXiv:2510.08191*, 2025.
- [6] Kang Chen, Yaoning Wang, Kai Xiong, Zhuoka Feng, Wenhe Sun, Haotian Chen, and Yixin Cao. Do llms signal when they’re right? evidence from neuron agreement, 2025. URL <https://arxiv.org/abs/2510.26277>.
- [7] Scott Geng, Dutch Hansen, and Jerry Li. Weak-to-strong generalization is nearly inevitable (in linear models). *arXiv preprint arXiv:2605.05742*, 2026.
- [8] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [9] Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning, 2025. URL <https://arxiv.org/abs/2504.11456>.
- [10] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [11] Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models. *arXiv e-prints*, pages arXiv–2501, 2025.
- [12] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International conference on learning representations*, volume 2024, pages 32808–32824, 2024.
- [13] Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. Process reward models that think. *arXiv preprint arXiv:2504.16828*, 2025.
- [14] Sanghwan Kim, Rui Xiao, Stephan Alaniz, Yongqin Xian, and Zeynep Akata. Training-free uncertainty guidance for complex visual tasks with mllms. *arXiv preprint arXiv:2510.00705*, 2025.
- [15] Hao Lang, Fei Huang, and Yongbin Li. Selective weak-to-strong generalization, 2025. URL <https://arxiv.org/abs/2511.14166>.
- [16] Shiye Lei, Zhihao Cheng, and Dacheng Tao. A step back: Prefix importance ratio stabilizes policy optimization, 2026. URL <https://arxiv.org/abs/2601.22718>.
- [17] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 7871–7880, 2020.
- [18] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, pages 39578–39601, 2024.

- [19] Huanyu Liu, Jia Li, Yihong Dong, Chang Yu, Taozhi Chen, Lecheng Wang, Yongding Tao, Bin Gu, and Ge Li. Evocot: Overcoming the exploration bottleneck in reinforcement learning. *arXiv preprint arXiv:2508.07809*, 2025.
- [20] Junteng Liu, Yuanxiang Fan, Zhuo Jiang, Han Ding, Yongyi Hu, Chi Zhang, Yiqi Shi, Shitong Weng, Aili Chen, Shiqi Chen, Yunan Huang, Mozhi Zhang, Pengyu Zhao, Junjie Yan, and Junxian He. Synlogic: Synthesizing verifiable reasoning data at scale for learning logical reasoning and beyond, 2025. URL <https://arxiv.org/abs/2505.19641>.
- [21] Yi Liu, Guoyin Wang, Shicheng Li, Feifan Song, and Xu Sun. ATLANTIS: Weak-to-strong learning via importance sampling. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1042–1052, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.52. URL <https://aclanthology.org/2025.acl-long.52/>.
- [22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [23] Yuxiao Qu, Amrith Setlur, Virginia Smith, Ruslan Salakhutdinov, and Aviral Kumar. Pope: Learning to reason on hard problems via privileged on-policy exploration. *arXiv preprint arXiv:2601.18779*, 2026.
- [24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [25] Amrith Setlur, Zijian Wang, Andrew Cohen, Paria Rashidinejad, and Sang Michael Xie. Reuse your flops: Scaling rl on hard problems by conditioning on very off-policy prefixes. *arXiv preprint arXiv:2601.18795*, 2026.
- [26] Amrith Setlur, Zijian Wang, Andrew Cohen, Paria Rashidinejad, and Sang Michael Xie. Reuse your flops: Scaling rl on hard problems by conditioning on very off-policy prefixes, 2026. URL <https://arxiv.org/abs/2601.18795>.
- [27] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [28] Seamus Somerstep, Felipe Maia Polo, Moulinath Banerjee, Yaacov Ritov, Mikhail Yurochkin, and Yuekai Sun. A statistical framework for weak-to-strong generalization. In *ICML 2024 Next Generation of AI Safety Workshop*, 2024.
- [29] Yiliu Sun, Zicheng Zhao, Yang Wei, Yanfang Zhang, and Chen Gong. Well begun, half done: Reinforcement learning with prefix optimization for llm reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 33144–33152, 2026.
- [30] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13003–13051, 2023.
- [31] Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [32] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103, 2008.
- [33] Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. *arXiv preprint arXiv:2211.00053*, 2022.
- [34] Changyi Xiao, Mengdi Zhang, and Yixin Cao. Bnpo: Beta normalization policy optimization, 2025. URL <https://arxiv.org/abs/2506.02864>.
- [35] Caijun Xu, Changyi Xiao, Zhongyuan Peng, Xinrun Wang, and Yixin Cao. Scaler: Synthetic scalable adaptive learning environment for reasoning. *arXiv preprint arXiv:2601.04809*, 2026.
- [36] Jianhao Yan, Yafu Li, Zican Hu, Zhi Wang, Ganqu Cui, Xiaoye Qu, Yu Cheng, and Yue Zhang. Learning to reason under off-policy guidance. *Advances in Neural Information Processing Systems*, 38:117157–117186, 2026.
- [37] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei

Zhang, Jianxin Ma, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2 technical report. [arXiv preprint arXiv:2407.10671](https://arxiv.org/abs/2407.10671), 2024.

- [38] Wei Yao, Wenkai Yang, Ziqiao Wang, Yankai Lin, and Yong Liu. Revisiting weak-to-strong generalization in theory and practice: Reverse kl vs. forward kl. In [Findings of the Association for Computational Linguistics: ACL 2025](#), pages 2860–2888, 2025.
- [39] Linhao Yu, Tianmeng Yang, Siyu Ding, Renren Jin, Naibin Gu, Xiangzhao Hao, Shuaiyi Nie, Deyi Xiong, Weichong Yin, Yu Sun, and Hua Wu. Knowrl: Boosting llm reasoning via reinforcement learning with minimal-sufficient knowledge guidance, 2026. URL <https://arxiv.org/abs/2604.12627>.
- [40] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. [Advances in Neural Information Processing Systems](#), 38:113222–113244, 2026.
- [41] Yige Yuan, Teng Xiao, Shuchang Tao, Xue Wang, Jinyang Gao, Bolin Ding, and Bingbing Xu. Incentivizing strong reasoning from weak supervision, 2026. URL <https://arxiv.org/abs/2505.20072>.
- [42] Shaoxiong Zhan, Yanlin Lai, Ziyu Lu, Dahua Lin, Ziqing Yang, and Fei Tan. Mathsmith: Towards extremely hard mathematical reasoning by forging synthetic problems with a reinforced policy, 2026. URL <https://arxiv.org/abs/2508.05592>.
- [43] Zhaoyang Zhang, Shuli Jiang, Yantao Shen, Yuting Zhang, Dhananjay Ram, Shuo Yang, Zhuowen Tu, Wei Xia, and Stefano Soatto. Reinforcement-aware knowledge distillation for llm reasoning, 2026. URL <https://arxiv.org/abs/2602.22495>.